KULLANIM VE TEKNİK KILAVUZ

Bütçe Hava Durumu

Harcamalarını takip et, bütçenin durumunu hava durumu gibi gör.

SÜRÜM
v1.8

TARİH
5 Ekim 2026

KAYNAK
github.com/Yaren-Dogan/expense-tracking

İÇERİK
Kullanım · kurulum · mimari · veritabanı · API

Bu doküman her güncellemede yeniden üretilir. Veritabanı bölümü, veritabanının içine yazılmış açıklamalardan otomatik oluşturulur; değişiklik geçmişi son bölümdedir.

İçindekiler

1 **Uygulama nedir?**
- 1.1 Özellikler
- 1.2 Teknoloji özeti

2 **Kullanım kılavuzu**
- 2.1 Kayıt ve giriş
- 2.2 Bütçe ve hava durumu
- 2.3 Harcama eklemek
- 2.4 Harcama listesi ve son 7 gün
- 2.5 Geçmiş aylar
- 2.6 Ortak hane
- 2.7 Görevler
- 2.8 Bildirimler
- 2.9 Ay değerlendirmesi
- 2.10 Birikim planlayıcı
- 2.11 Bütçe asistanı
- 2.12 Telefona kurmak

3 **Kurulum ve çalıştırma**
- 3.1 Gerekenler
- 3.2 İlk kurulum
- 3.3 Güncelleme
- 3.4 Servisler ve adresler
- 3.5 Veritabanına bakmak (Adminer)
- 3.6 Ortam değişkenleri (.env)

4 **Mimari**
- 4.1 Bir isteğin yolculuğu
- 4.2 Defter (ledger) modeli
- 4.3 Hava durumu kuralları
- 4.4 Tasarım kararları

5 **Güvenlik**

6 **Yapay zekâ**
- 6.1 Cümleyle harcama ekleme
- 6.2 Anlamsal arama (RAG)
- 6.3 Ajanlar

7 **Ajanlar**
- 7.1 Agent nedir, nasıl çalışır?
- 7.2 Ajan haritası
- 7.3 Bütçe asistanı
- 7.4 Ay değerlendirmesi ajanı
- 7.5 Birikim planlayıcı ajanı
- 7.6 Geliştirme ajanları (Claude Code)

8 **Veritabanı**
- 8.1 İlişki diyagramı
- 8.2 Tablolar ve kolonlar

9 **API referansı**

10 **Test ve kalite**
- 10.1 Geliştirme komutları

11 **Değişiklik geçmişi**

1 Uygulama nedir?

Klasik harcama uygulamaları grafik ve tablo gösterir; insanlar bir süre sonra bakmayı bırakır. Bütçe Hava Durumu, bütçenin bugünkü durumunu herkesin bir bakışta anladığı bir dille anlatır: **hava durumu**. Bugün güneşli mi, fırtına mı geliyor?

Güneşli	Parçalı bulutlu	Bulutlu	Yağmurlu	Fırtına
Günlük payının yarısından azını harcadın	Payının içindesin	Payının sınırındasın	Payını aştın	Payının 1,5 katından fazlası

1.1 Özellikler

Bugünün havası	Bugünkü harcaman günlük payınla karşılaştırılır; "gönül rahatlığıyla harcayabileceğin" tutar gösterilir.
Son 7 gün ve ay sonu tahmini	Her günün havası ve bu hızla ayın nasıl kapanacağı.
Geçmiş aylar	Önceki aylara gidip harcama ve bütçe girilebilir; ayın kapanış havası görünür.
Ortak hane	Eşler ve aileler aynı bütçeyi ve harcama listesini paylaşır. Herkes sadece kendi girdiğini siler.
Görevler ve bildirimler	Faturalar hane üyelerine atanır, son gün yaklaşınca bildirim gider; "Ödendi" deyince harcama eklenir.
Ay değerlendirmesi	Ay bitince değerlendirme ajanı ayı inceler: öne çıkanlar ve bir tavsiye, hane üyelerine bildirimle.
Birikim planlayıcı	"6 ayda ₺20.000" gibi bir hedef için planlayıcı ajan hangi kalemlerden kısılacağını ve aylık bütçeyi önerir; onaylayınca uygulanır.
Yapay zekâ	Cümleyle harcama ekleme, anlamsal arama ve verilerinden cevap veren bütçe asistanı.
Her cihazda	Web ve mobil uyumlu PWA; telefonda "Ana ekrana ekle" ile uygulama gibi çalışır.

1.2 Teknoloji özeti

BACKEND
.NET 9, ASP.NET Core, Clean Architecture, MediatR (CQRS), FluentValidation, EF Core 9

VERİTABANI
PostgreSQL 17 + pgvector (vektör arama)

KİMLİK
ASP.NET Core Identity, HttpOnly cookie oturumu

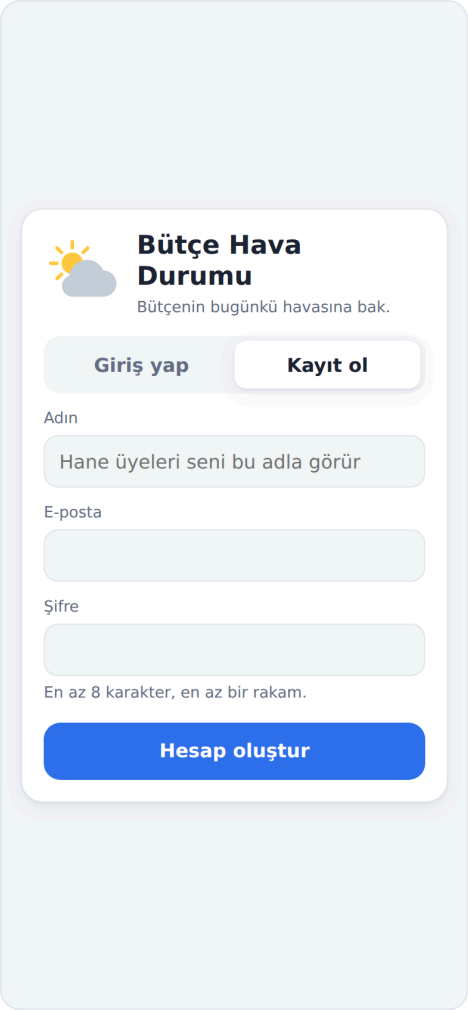
YAPAY ZEKÂ
Microsoft.Extensions.AI; Google Gemini (ücretsiz) ya da yerel Ollama

FRONTEND
React 19, TypeScript, Vite, PWA

ÇALIŞTIRMA
Docker Compose (Postgres, API, web, Adminer)

2 Kullanım kılavuzu

2.1 Kayıt ve giriş



Uygulamayı ilk açtığında giriş ekranı gelir. **Kayıt ol** sekmesinde adını, e-postanı ve şifreni gir.

- **Adın**, hane üyelerinin seni göreceği addır.
- **Şifre** en az 8 karakter olmalı ve en az bir rakam içermeli.
- 5 kez hatalı şifre girilirse hesap 5 dakika kilitlenir.
- Oturum 14 gün açık kalır; her kullanımda süre uzar. Sağ üstteki **Çıkış** ile kapatılır.

> Hesaplardan önce bu tarayıcıda harcama girdiysen, ilk kayıtta bu harcamalar hesabına taşınır ve ekranda kaç harcamanın taşındığı yazar.

2.2 Bütçe ve hava durumu



Hava durumu için önce o ayın bütçesi gerekir. İlk açılışta **Aylık bütçeni belirle** düğmesine bas; sonra değiştirmek için üstteki **Bütçe: ₺...** etiketine dokun.

Ana ekranın üst kısmı bugünün havasıdır:

- **Gönül rahatlığıyla harcayabileceğin:** havayı bozmadan bugün daha ne kadar harcayabileceğin.
- **Çubuk:** bugünkü günlük payının ne kadarını kullandığın.
- **Günlük pay:** kalan bütçenin kalan günlere bölünmüş hali. Dün az harcadıysan bugünkü payın büyür.

Hesap kuralları teknik bölümde, hava durumu kurallarında anlatılır.

2.3 Harcama eklemek

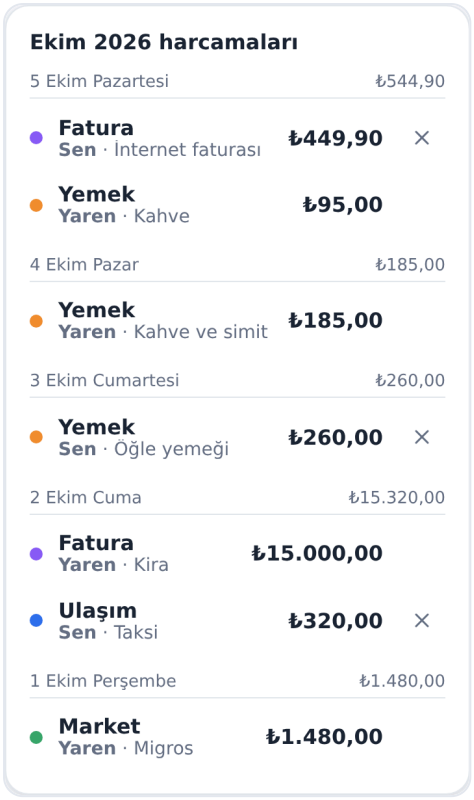


Tutarı yaz (85,50 ya da 1.250,75 gibi Türkçe yazım kabul edilir), kategoriyi seç, istersen not ekle ve **Ekle**'ye bas. Tarih bugündür; geçmiş bir gün de seçilebilir, gelecek seçilemez.

Cümleyle ekleme: üstteki kutuya "dün akşam kahveye 85 lira verdim" yazıp **Anla**'ya bas. Yapay zekâ tutarı, kategoriyi, notu ve tarihi forma doldurur. Kaydetmez: kontrol edip **Ekle**'ye sen basarsın.

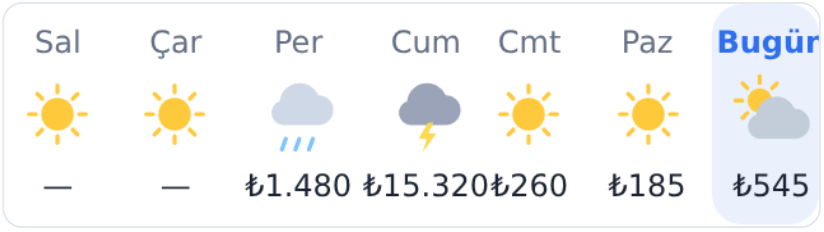
> Çift dokunma ya da bağlantı kopması harcamayı iki kez kaydetmez (idempotency).

2.4 Harcama listesi ve son 7 gün



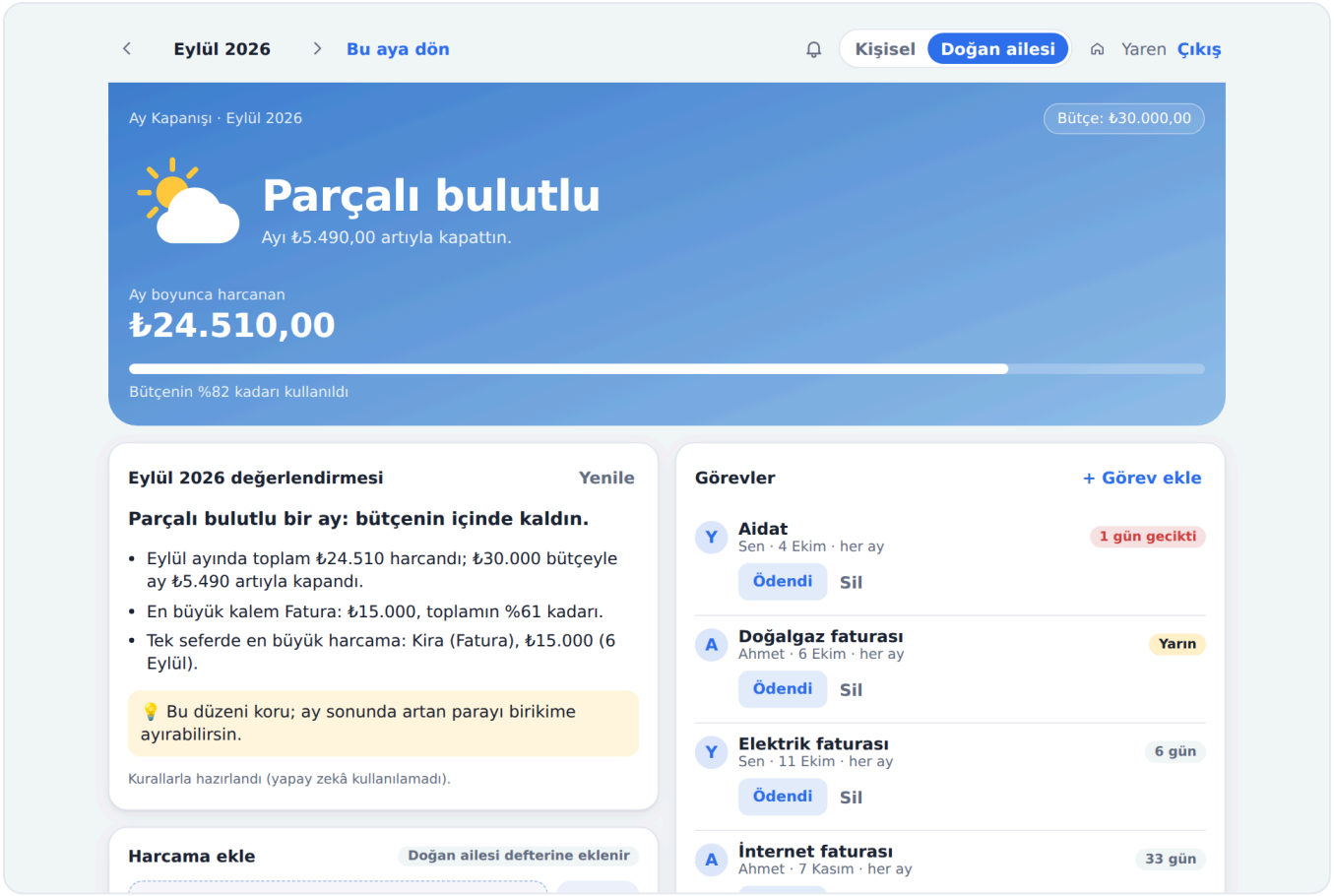
Liste, görüntülenen ayın harcamalarını güne göre gruplar. Hanedeysen her harcamada kimin girdiği yazar ("Sen" ya da üyenin adı).

Bir harcamayı silmek için yanındaki **×** işaretine bas. Hanede herkes tüm harcamaları görür ama sadece kendi girdiğini silebilir; başkasının harcamasında silme işareti çıkmaz.



Son 7 günün havası her günün havasını ve o gün harcananı gösterir. Altındaki **Ay sonu tahmini**, bu hızla ayın kaç lirayla kapanacağını söyler.

2.5 Geçmiş aylar



Sol üstteki **‹** okuyla önceki aylara gidilir; **›** ileri götürür, gelecek aya geçilemez. **Bu aya dön** ile hemen bugüne dönülür.

- Geçmiş bir ayın havası, **ayın kapanışıdır**: toplam harcamanın bütçeye oranı ("Ayı ₺5.490 artıyla kapattın").
- Geçmiş aya harcama eklerken tarih varsayılan olarak o ayın son günüdür.
- Geçmiş ayların bütçesi de girilebilir ve değiştirilebilir.

2.6 Ortak hane



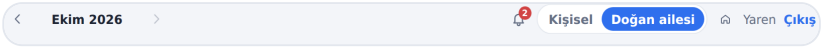


Evli çiftler ve aileler ortak bir **defter** tutar: aynı bütçe, aynı harcama listesi, aynı hava durumu. Üst çubuktaki **Kişisel** düğmesi hane panelini açar.

1. **Hane kur:** bir ad ver (örn. "Doğan ailesi"). Kurucu olursun ve 8 karakterlik bir **davet kodu** alırsın.
2. Kodu eşine ya da ailene gönder (**Kopyala** düğmesi).
3. **Katılacak kişi** kodu girip **Devam**'a basar. Uygulama sorar: **"Aynı haneden misiniz?"** ve hanenin adını, üyelerini gösterir. **Evet, katıl** ile katılır.

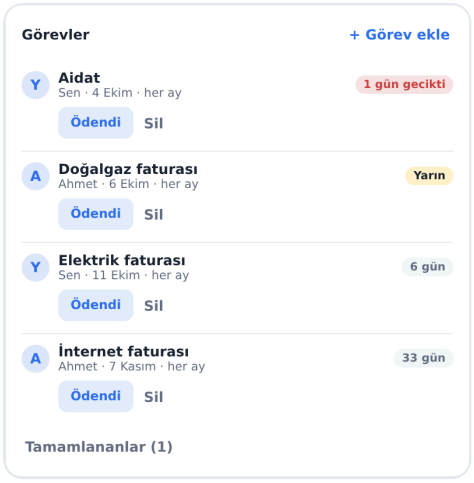
> Katılınca eski kişisel harcamaların sende kalır, hanede görünmez. Haneden ayrılınca kişisel defterine dönersin; hane harcamaları hanede kalır.

Kişisel ↔ hane geçişi: Hanedeyken üst çubukta **Kişisel | Hane adı** seçicisi çıkar. Haneden ayrılmadan kişisel defterine geçersin: kendi bütçen, kendi harcamaların, kendi görevlerin. Harcama ekle kartının köşesinde hangi deftere ekleneceği yazar ("Kişisel defterine eklenir"). Kişisel defterini hane üyeleri görmez. Seçim bu cihazda hatırlanır; hane ayarları (davet kodu, üyeler, ayrılma) seçicinin yanındaki ev simgesindedir.



Kural	Açıklama
Tek hane	Bir kişi aynı anda tek hanede olabilir.
En fazla 10 üye	Kötüye kullanım sınırı.
Bütçe	Her üye değiştirebilir (ortak karar).
Silme	Herkes sadece kendi girdiği harcamayı siler.
Kurucu	Üye çıkarır, davet kodunu yeniler (eski kod çalışmaz). Ayrılırsa kuruculuk en eski üyeye geçer.

2.7 Görevler



Faturaları ve ödemeleri kişilere dağıt: "Elektrik faturası → Yaren", "İnternet faturası → Ahmet". **+ Görev ekle** ile görevin adını, kimin yapacağını, son günü ve ödenince kaydedilecek kategoriyi seç.

- **Her ay tekrarla** açıksa, görev tamamlanınca gelecek ayın görevi aynı gün kendiliğinden açılır (31'i kısa aylarda ayın son gününe çekilir).
- Sağdaki etiket kalan süreyi gösterir: Yarın 1 gün gecikti.
- Görevi **alan** ve **veren** tamamlayabilir; **veren** ve **hane kurucusu** silebilir. Tamamlanmış görevi **alan** da listeden silebilir.
- Hanesi olmayan kullanıcı da görevleri kişisel hatırlatıcı olarak kullanabilir.
- Haneden ayrılan üyenin açık görevleri kurucuya geçer.

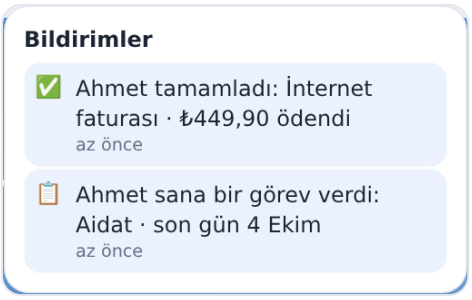


Ödendi: düğmeye basınca tutar sorulur. Tutarı yazarsan ödeme, görevin kategorisinde (varsayılan Fatura) hane harcamalarına eklenir ve bütçe havası güncellenir. Tutarı boş bırakırsan görev sadece tamamlanır.

Görev Anıl'ınsa ve "Ödendi"yi görevi veren Yaren işaretlediyse görev yine Anıl'ın kalır: harcama **Anıl'ın adıyla** kaydedilir (Anıl silebilir), tamamlananlar listesinde "Anıl · Yaren işaretledi" yazar ve Anıl'a bildirim gider.

Tamamlanan görevler kartın altındaki **Tamamlananlar** listesinde, kimin ödediği ve tutarıyla görünür.

2.8 Bildirimler

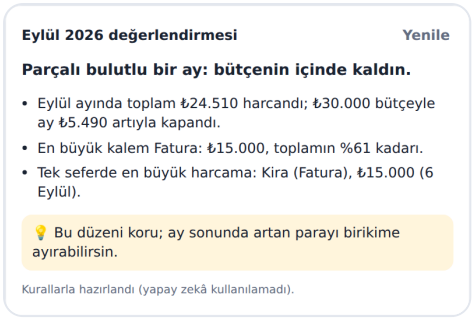


Üst çubuktaki zil, okunmamış bildirim sayısını gösterir. Zile basınca panel açılır ve hepsi okundu sayılır. Uygulama açıkken bildirimler dakikada bir yenilenir.

Ne zaman	Kime
Görev verildi	Görevi alan (kendine verdiysen gitmez)
Son güne 3 gün kaldı	Görevi alan
Son gün geçti	Görevi alan
Görev tamamlandı	Görevi veren (ve başkası yaptıysa görevi alan)
Üye haneden ayrıldı	Görevleri devralan kurucu
Ay değerlendirmesi hazır	Hanenin tüm üyeleri
Birikim planı uygulandı	Planı uygulayan dışındaki üyeler

> Her hatırlatma bir kez gider. Bildirimler şimdilik uygulama içindedir; telefona bildirim (push) yoktur.

2.9 Ay değerlendirmesi



Her ay bittiğinde **değerlendirme ajanı** o ayı inceler ve kısa bir değerlendirme yazar: ayı özetleyen bir başlık, öne çıkan 2-4 gerçek (toplam, bütçe sonucu, en çok artan kategori, en büyük harcama, hanede kimin ne kadar girdiği) ve önümüzdeki ay için bir tavsiye. Hazır olunca hane üyelerine bildirim gider.

- Geçmiş bir aya gidince değerlendirme kartı en üstte görünür. Henüz yoksa **Değerlendir** ile hemen yazdırabilirsin; **Yenile** yeniden yazar.
- Yeni ayın ilk haftasında geçen ayın değerlendirmesi ana ekranda da görünür.
- Rakamların hepsi kodda hesaplanır; ajan sadece hangilerinin önemli olduğunu seçer ve başlıkla tavsiyeyi yazar.
- Yapay zekâ kullanılamazsa değerlendirme kurallarla yazılır; kartın altında hangisinin yazdığı belirtilir.
- Ajanın nasıl çalıştığı **Ajanlar** bölümünde örnek bir akışla anlatılır.

2.10 Birikim planlayıcı



Bir hedefin mi var? **Birikim planlayıcı** kartına tutarı, kaç ayda biriktirmek istediğini ve istersen amacını yaz; **Plan yap**'a bas. **Planlayıcı ajan** son ayların harcamalarına bakar ve:

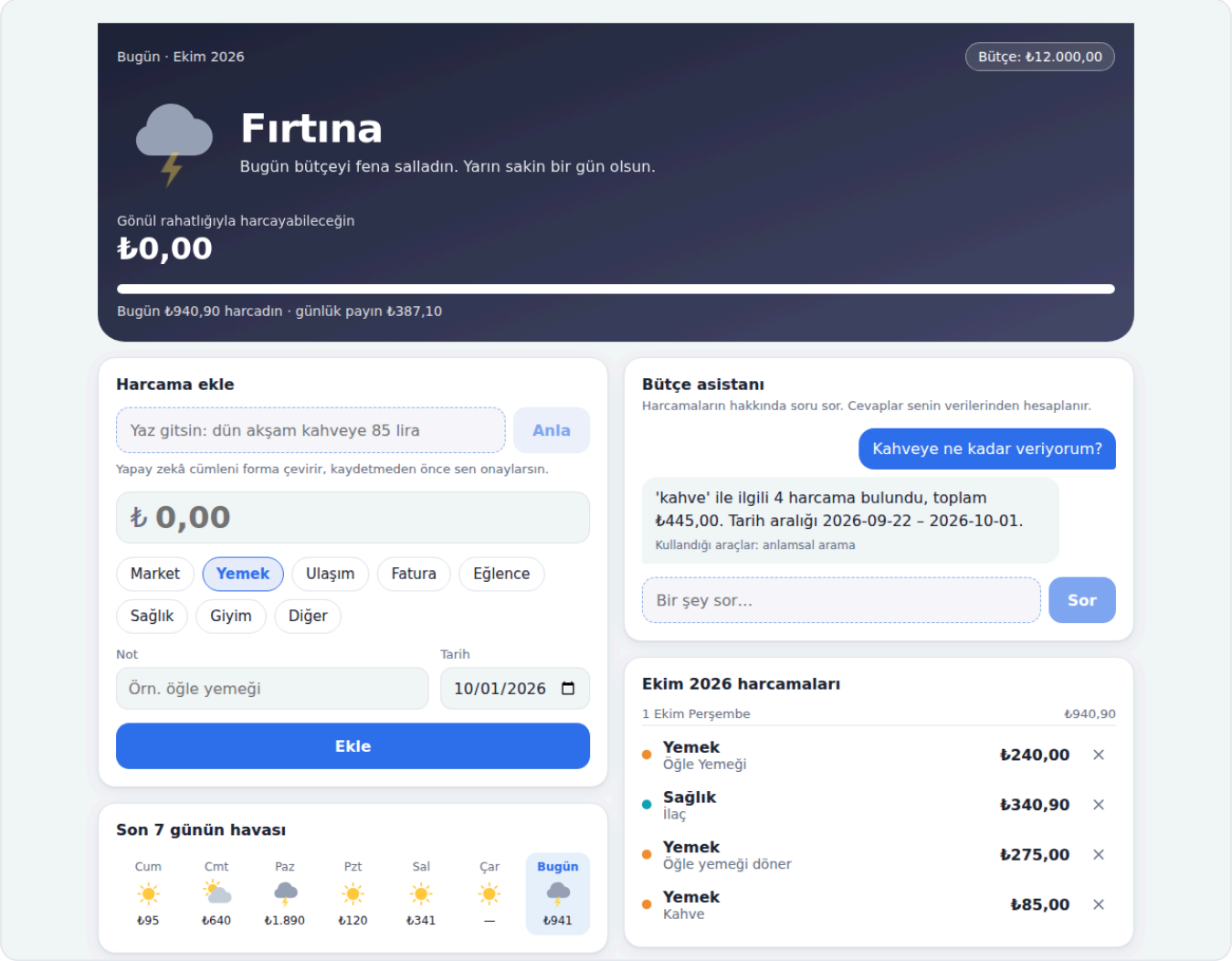
- hangi kategorilerden kısabileceğini seçer (her biri için neden ve ne kadar: az, orta, çok),
- her kategori için şu anki aylık ortalamayı ve hedefi gösterir,
- planın süresince her ay için yeni bütçeyi hesaplar (şu anki ortalama − aylık birikim),
- bir alışkanlık önerisi ekler.

Fatura ve sağlık asla kısılmaz, hiçbir kategori yarısından fazla kısılmaz. Hedef istediğin sürede mümkün değilse plan uzatılır ve bu açıkça yazılır.

> Plan önce sadece bir öneridir. **Planı uygula** ve onaydan sonra planın aylarındaki bütçeler yazılır (hanede herkes için). Plan, yapıldığı ayın ertesi ayından başlar. Hesabın nasıl yapıldığı **Ajanlar** bölümünde örnekle anlatılır.

2.11 Bütçe asistanı

Sağdaki **Bütçe asistanı** kutusuna Türkçe soru sor: "Bu ay bütçem nasıl gidiyor?", "Geçen ay en çok nereye harcadım?", "Kahveye ne kadar veriyorum?". Asistan cevabı uydurmaz; senin verilerini okuyan araçları çalıştırır ve cevabın altında hangi araçları kullandığını gösterir. Sayılar her zaman kodda hesaplanır, modelden gelmez.



> Yapay zekâ özellikleri için bir Gemini API anahtarı gerekir (kurulum bölümü). Anahtar yoksa uygulamanın geri kalanı çalışır; bu özellikler "yapılandırılmamış" mesajı verir. Ücretsiz katmanda gönderilen veriler Google tarafından model geliştirmede kullanılabilir; gerçek kişisel veri yerine örnek veriyle denemek iyi olur.

2.12 Telefona kurmak

Uygulama bir PWA'dır. Telefonda tarayıcıdan aç; Android/Chrome'da menüden **Ana ekrana ekle**, iPhone/Safari'de **Paylaş → Ana Ekrana Ekle**. Simgesiyle açılır ve tam ekran çalışır.

3 Kurulum ve çalıştırma

3.1 Gerekenler

Sadece **Docker Desktop**. .NET, Node ya da PostgreSQL kurmaya gerek yok; hepsi konteynerde çalışır.

3.2 İlk kurulum

1. Repoyu klonla: `git clone https://github.com/Yaren-Dogan/expense-tracking`
2. Yapay zekâ için ücretsiz Gemini API anahtarı al: `aistudio.google.com/apikey`
3. `.env.example` dosyasını `.env` olarak kopyala ve `AI_API_KEY=` satırını doldur. `.env` git'e eklenmez.
4. Çalıştır: `docker compose up -d --build`
5. Tarayıcıda aç: `http://localhost:8080`

3.3 Güncelleme

```
cd D:\Projects\expense-tracking
git pull
docker compose up -d --build
```

Veritabanı değişiklikleri (migration'lar) API açılırken otomatik uygulanır; veri kaybolmaz. Güncellemeden sonra eski ekran görünürse tarayıcıda **Ctrl+Shift+R** ile tam yenile.

3.4 Servisler ve adresler

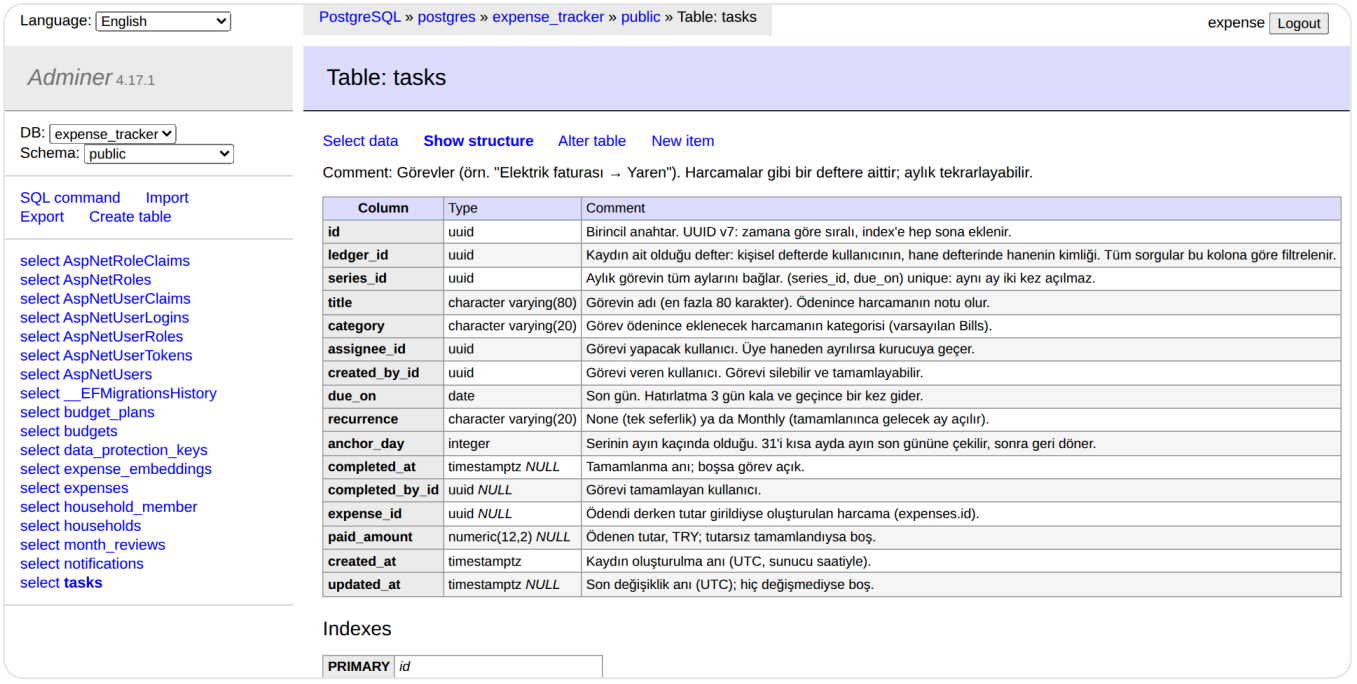
Servis	Adres	Ne işe yarar
web	`http://localhost:8080`	Uygulama (nginx, React PWA). `/api` isteklerini API'ye iletir.
api	`http://localhost:5080`	.NET API. Normalde web üzerinden kullanılır.
postgres	`localhost:5432`	PostgreSQL 17 + pgvector. Veriler `pgdata` volume'ünde kalıcıdır.
adminer	`http://localhost:8081`	Veritabanını tarayıcıdan görmek için panel.
ollama (opsiyonel)	—	`docker compose --profile ollama up -d` ile yerel model (~7 GB indirir).

3.5 Veritabanına bakmak (Adminer)

`http://localhost:8081` adresinde şu bilgilerle giriş yap:

Sistem	PostgreSQL
Sunucu	`postgres`
Kullanıcı	`expense`
Parola	`expense_dev` (sadece yerel geliştirme için; `.env`'de `POSTGRES_PASSWORD` ile değiştirilebilir)
Veri tabanı	`expense_tracker`

Soldan bir tabloya tıklayınca yapı görünümünde tablonun ve her kolonun Türkçe açıklaması görünür ("Comment"/"Yorum"):

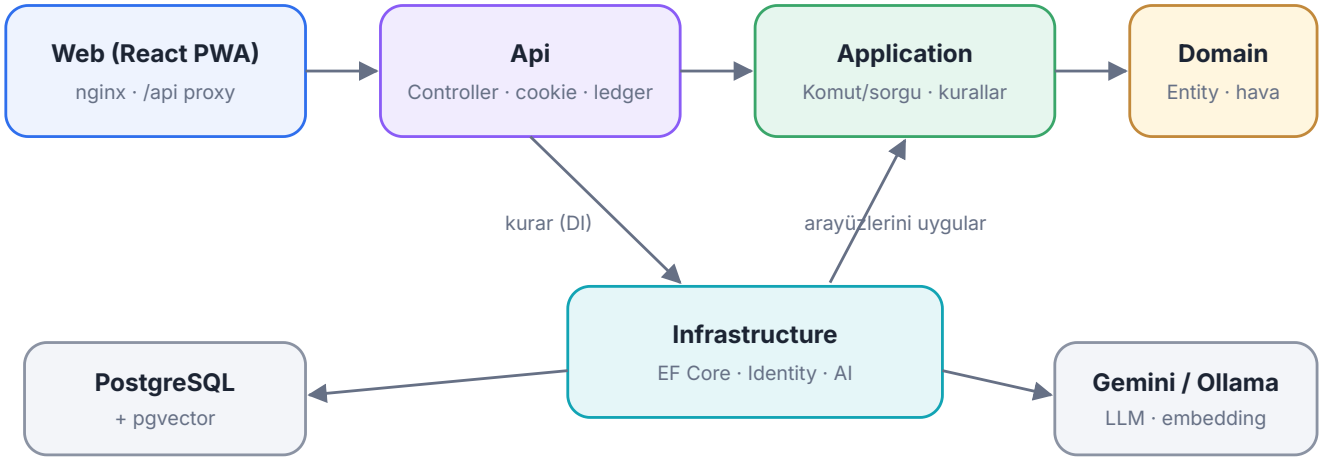


3.6 Ortam değişkenleri (`.env`)

Değişken	Varsayılan	Açıklama
`AI_API_KEY`	—	Gemini API anahtarı. Boşsa yapay zekâ uçları 503 döner.
`CHAT_MODEL`	`gemini-3.5-flash-lite`	Sohbet/çıkarım modeli.
`EMBEDDING_MODEL`	`gemini-embedding-001`	Anlamsal arama vektör modeli.
`AI_PROVIDER`	`OpenAICompatible`	`Ollama` ile yerel model.
`AGENT_MODE`	`Planned`	Asistanın çalışma modu (yapay zekâ bölümü).
`POSTGRES_PASSWORD`	`expense_dev`	Veritabanı şifresi. İnternete açılacaksa mutlaka değiştir.

4 Mimari

Backend **Clean Architecture** ile dört katmana ayrılır. Bağımlılıklar sadece içe doğrudur: Domain hiçbir şeye, Application sadece Domain'e bağlıdır. Veritabanı, kimlik ve yapay zekâ sağlayıcısı gibi dış dünya detayları en dış katmandadır; iş kuralları onlardan habersizdir ve tek başına test edilir.



Oklar "bağımlıdır" anlamındadır. Infrastructure, Application'daki arayüzleri (IAppDbContext, IUserDirectory, IChatClient...) uygular.

Katman	İçerik	Örnek
Domain	Entity'ler, iş kuralları, hava hesabı. Hiçbir pakete bağlı değil.	`Expense.Create`, `Household.Join`, `HouseholdTask.Complete`, `BudgetWeather.ForDay`
Application	Her kullanım senaryosu kendi klasöründe: komut/sorgu + handler + validator.	`Features/Tasks/Commands/CompleteTask`
Infrastructure	EF Core + PostgreSQL, Identity kullanıcıları, model istemcileri, arka plan işleri.	`AppDbContext`, `PgvectorExpenseSearch`, `TaskReminderScheduler`
Api	İnce controller'lar, oturum, hata cevapları (ProblemDetails), istek sınırı.	`TasksController`, `LedgerMiddleware`

4.1 Bir isteğin yolculuğu

Örnek: Ahmet bir görevi "Ödendi" diye işaretliyor (`POST /api/tasks/{id}/complete`, tutar 449,90).

1. **nginx** isteği API'ye iletir. Tarayıcı oturum cookie'sini (`bhd.auth`) kendiliğinden gönderir.
2. **Kimlik doğrulama** cookie'den kullanıcıyı çıkarır; oturum yoksa `401`.
3. **LedgerMiddleware** aktif defteri belirler: Ahmet bir hanedeyse hane kimliği.
4. **Controller** iş yapmaz: `CompleteTaskCommand`'ı MediatR ile gönderir.
5. **ValidationBehavior** komutu FluentValidation ile doğrular (tutar 0'dan büyük, en fazla 2 ondalık); hata varsa alan bazlı `400`.
6. **Handler** görevi aktif defterde bulur, yetkiyi kontrol eder (görevi alan ya da veren mi?), harcamayı oluşturur, `task.Complete(...)` ile görevi kapatır, aylık görevin gelecek ayını ekler, görevi verene bildirim yazar.
7. **SaveChanges** hepsini tek transaction'da yazar. İki kişi aynı anda bastıysa unique index ikinciyi reddeder → `409`.

4.2 Defter (ledger) modeli

Her harcama, bütçe ve görev bir **deftere** aittir (`ledger_id`). Kişisel defterin kimliği kullanıcının kimliği, hane defterinin kimliği hanenin kimliğidir. Kullanıcının **aktif defteri**: bir hanedeyse hane defteri, değilse kişisel defteri. Tüm okuma ve yazmalar aktif deftere göre filtrelenir; `user_id` sadece kaydı kimin girdiğini gösterir.



Haneye katılmak ya da ayrılmak veri kopyalamaz; sadece hangi defterin okunup yazıldığı değişir.

Hanedeki kullanıcı istediğinde kişisel defterine geçebilir: istemci `X-Ledger: personal` header'ı gönderir, `LedgerMiddleware` aktif defteri kullanıcının kendi kimliği yapar. Hane defteri her zaman üyelikten belirlenir, header'dan asla alınmaz; bu yüzden geçiş başka birinin verisine yol açamaz.

Bu tasarım sayesinde "katılınca eski harcamalar kişisel kalsın, ayrılınca hane harcamaları hanede kalsın" kuralı ek tablo ya da kopyalama olmadan sağlanır. İzolasyon iki katmanlıdır: her sorguda açık `LedgerId` koşulu ve EF Core'un global query filter'ı (unutulan koşula karşı ikinci savunma).

4.3 Hava durumu kuralları

Gün `d`, ayın bütçesi `B` olsun:

`S0`	Ayın başından `d`'den önceki güne kadar harcanan toplam
`R`	`d` dahil ayın kalan gün sayısı
Günlük pay	`A = max(0, (B - S0) / R)`
`T`	`d` günü harcanan toplam; oran `r = T / A`
Gönül rahatlığıyla harcanabilir	`max(0, A - T)`

Hava	Koşul
Güneşli	`r ≤ 0,5`
Parçalı bulutlu	`0,5 < r ≤ 0,9`
Bulutlu	`0,9 < r ≤ 1,0`; ya da bütçe bitti ama bugün harcama yok
Yağmurlu	`1,0 < r ≤ 1,5`
Fırtına	`r > 1,5`; ya da bütçe bitmişken harcama yapıldı

Ay sonu tahmini: günlük ortalama (`ay içi toplam / d.Day`) × ayın gün sayısı; havası `tahmin / B` oranına aynı eşiklerle uygulanır. Geçmiş bir ay son gününden bakıldığı için tahmin, gerçek toplamdır (ayın kapanışı). "Bugün" Europe/Istanbul saat dilimine göredir.

4.4 Tasarım kararları

Karar	Neden
Repository yok, handler'lar `IAppDbContext` ile LINQ yazar	EF Core zaten unit of work + repository; tekrar sarmak sadece kod çoğaltır.
Durum değişikliği entity metodlarıyla (`Complete`, `Join`, `ChangeAmount`)	Kurallar tek yerde; geçersiz durum oluşturulamaz.
Kritik tekillikler unique index ile	"Önce kontrol et sonra yaz" yarışta bozulur; veritabanı garantisi bozulmaz.
Sadece unique ihlali yakalanır	Diğer veritabanı hataları gizlenmez, yukarı fırlar.
`TimeProvider` + saat dilimi, `DateTime.Now` yok	Testlerde zaman sabitlenir; gece yarısı hataları olmaz.
UUID v7 birincil anahtar	Zamana göre sıralı: index'e sona eklenir, tahmin edilemez.
Para `decimal`, tarih `DateOnly`	Kuruş yuvarlama hatası ve saat dilimi karışıklığı olmaz.
Yetki sunucuda hesaplanır (`canComplete`, `canDelete`)	Arayüz sadece izin verilen düğmeyi gösterir; asıl kontrol API'dedir.
Log'lar `LoggerMessage` + sabit EventId	Yapısal, hızlı ve aranabilir loglar; kullanıcı içeriği loglanmaz.

5 Güvenlik

Konu	Uygulama
Şifre saklama	ASP.NET Core Identity, PBKDF2 hash. Şifre düz metin olarak hiçbir yerde tutulmaz. En az 8 karakter + rakam.
Oturum	HttpOnly cookie: sayfa script'leri okuyamaz (XSS ile token çalınamaz). `SameSite=Strict` + sadece JSON kabul eden API: başka siteden istek (CSRF) çalışmaz. 14 gün, kayan süre.
Kaba kuvvet	5 hatalı şifrede hesap 5 dk kilitlenir. Kayıt/giriş uçlarında IP başına, davet kodu uçlarında kullanıcı başına dakikada 10 istek.
Model kotası	Model çağıran uçlar (asistan, cümleyle ekleme, değerlendirme, plan) kullanıcı başına dakikada 20 istekle sınırlı; tek kullanıcı herkesin kotasını tüketemez.
Oturum anahtarları	Cookie'leri şifreleyen anahtarlar veritabanında (`data_protection_keys`); güncellemeden sonra kimsenin oturumu kapanmaz.
Kullanıcı keşfi	Bilinmeyen e-posta ve yanlış şifre aynı cevabı alır; kayıtlı adresler bulunamaz.
Varsayılan kapalı	Her uç oturum ister (fallback policy). Açık olanlar sadece kayıt, giriş ve health.
Veri izolasyonu	Aktif deftere göre açık filtre + global query filter. Başka hanenin kaydına erişim `404` . Defter seçici (`X-Ledger: personal`) sadece kullanıcının kendi kişisel defterini açabilir; hane defteri üyelikten gelir.
Yetki	Başkasının harcamasını silmek, görevini tamamlamak `403` . Kurucu işlemleri sadece kurucu.
Veri sahiplenme	Eski cihaz verisi taşınırken kayıtlı bir kullanıcının ya da hanenin kimliği gönderilirse reddedilir.
Davet kodu	Karışan karakterler olmadan 8 karakter (~10^{12} olasılık), kriptografik rastgele; kurucu yenileyebilir.
Gizli bilgiler	API anahtarı sadece `.env` 'de (git'e girmez). Loglara kullanıcı cümlesi yazılmaz, sadece uzunluğu.
Yapay zekâ	Ajan araçları salt okunur ve kullanıcı/defter parametresi almaz; model başkasının verisini isteyemez. Harcama notları ve plan amacı talimat değil veri olarak ele alınır (prompt injection); kullanıcının metni talimatlara değil, etiketlenmiş kullanıcı mesajına girer. Modelin yazdığı metin rakam ya da bağlantı içeriyorsa kullanılmaz. Bütçeyi değiştiren tek yazma işlemi (plan uygulama) insan onayı ister.

6 Yapay zekâ

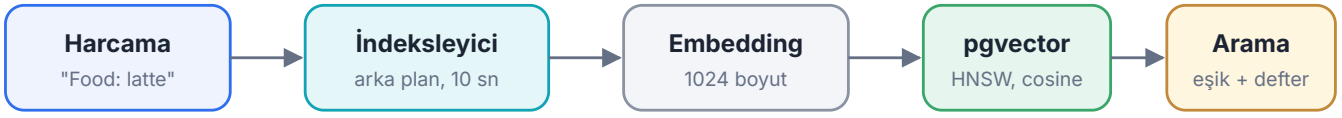
Özellik	Teknik	Uç
Cümleyle harcama ekleme	Structured output (JSON schema)	`POST /api/expenses/parse`
Anlamsal arama	RAG retrieval: embedding + pgvector	`GET /api/expenses/search?q=`
Bütçe asistanı	AI agent: araç çağırma + RAG	`POST /api/assistant/chat`
Ay değerlendirmesi	Proaktif agent: arka planda aylık çalışır, inceler, seçer	`POST /api/reviews/{ay}`
Birikim planlayıcı	Agent + kod hesabı + insan onayı	`POST /api/plans`

Kod `Microsoft.Extensions.AI` soyutlamalarına (`IChatClient` , `IEmbeddingGenerator`) yazılır; uygulama mantığı hangi sağlayıcının kullanıldığını bilmez. Varsayılan Google Gemini (ücretsiz, disk kullanmaz); `AI_PROVIDER=Ollama` ile yerel model.

6.1 Cümleyle harcama ekleme

- **LLM çıkarır, kod hesaplar.** Model tarih üretmez, sadece "kaç gün önce" (`daysAgo`) çıkarır; tarihi kod hesaplar.
- **JSON schema** ile kısıtlı çıktı: kategori bir enum, model listede olmayan değer üretemez.
- Few-shot örnekler, temperature 0, çıktı doğrulama (tutar ≤ 0 ise `422`).
- **İnsan onayı:** öneri forma dolar, kaydı kullanıcı yapar.

6.2 Anlamsal arama (RAG)



- Her harcama `"<Kategori>: <not>"` metni olarak vektöre çevrilir; "kahve" araması notunda kahve geçmeyen "Starbucks latte"yi de bulur.
- İndeksleme arka planda yapılır; model kapalıyken de harcama kaydedilir, model açılınca indekslenir.
- Gemini 3072 boyut döner; ilk 1024 boyuta kesilip yeniden normalize edilir (Matryoshka).
- Benzerlik eşiği modele göre: Gemini 0,68, bge-m3 0,5. Sonuçlar her zaman aktif deftere göre filtrelenir.

6.3 Ajanlar

Bütçe asistanı, ay değerlendirmesi ve birikim planlayıcı birer **ajandır**: bir döngü içinde araç seçer, sonucu görür ve karar verir. Nasıl çalıştıkları, araçları, güvenlik önlemleri ve örnek akışlarıyla bir sonraki bölümde anlatılır.

7 Ajanlar

Bu projede iki tür ajan var: **uygulama ajanları** kullanıcı için çalışır (bütçe asistanı, ay değerlendirmesi, birikim planlayıcı); **geliştirme ajanları** uygulamayı yazarken Claude Code içinde çalışır (mimar, geliştirici, test, inceleme, güvenlik, migration, doküman). Bu bölüm ikisini de, nasıl çalıştıklarıyla birlikte anlatır.

7.1 Agent nedir, nasıl çalışır?

Normal bir model çağrısı tek seferliktir: soru gider, cevap gelir. **Agent (ajan)** ise bir hedefe ulaşmak için hangi adımı atacağına kendisi karar veren, araç kullanan ve sonuca bakıp devam eden bir döngüdür.



Model sadece karar verir; aracı uygulama çalıştırır ve sonucu doğrular.

Parça	Ne işe yarar	Bu projede
Model	Karar verir: hangi araç, hangi parametre, ne zaman bitirilecek.	Gemini (ücretsiz) ya da yerel Ollama
Araçlar	Modelin dünyaya dokunduğu eller. Her birinin adı, açıklaması, parametre şeması var.	Hepsi salt okunur, kodda çalışır
Bağlam	Hedef, önceki adımlar ve araç sonuçları; model her turda bunları görür.	Sunucu durumsuz; her istek kendi döngüsü
Sınırlar	En fazla kaç tur, hangi araçlar, ne zaman insana sorulacağı.	3-5 tur, yazma işlemi insan onayıyla

Bu projenin ajan ilkeleri

- **Model karar verir, kod hesaplar.** Kullanıcının okuduğu her sayı kodda hesaplanır. Model hangi bilginin önemli olduğunu, hangi kategorinin kısılacağını seçer; rakam yazmaz. Modelin metni rakam ya da bağlantı içeriyorsa kullanılmaz.
- **Planned döngü.** Model her adımda JSON şemasıyla kısıtlanmış bir plan döner (eylem + parametre). Şema, küçük ve ücretsiz modellerde bile her zaman geçerli, çalıştırılabilir bir seçim üretilmesini sağlar. Modelin kendi araç çağırma özelliği (Native mod) güçlü modeller için seçenek olarak durur.
- **Kimlik koddan gelir.** Hiçbir araç kullanıcı ya da defter kimliğini modelden almaz; model başka bir hanenin verisini isteyemez.
- **Kullanıcı verisi talimat değildir.** Harcama notları ve plan amacı modele "veri" olarak, etiketli verilir (prompt injection).
- **İnsan onayı.** Veri değiştiren tek ajan sonucu (birikim planını uygulamak) kullanıcının onayıyla çalışır.
- **Kural tabanlı yedek.** Model yoksa, kota dolmuşsa ya da cevabı kullanılamazsa ajan kurallarla aynı işi yapar.
- **Test edilebilirlik.** Testler gerçek model çağırmaz; sahte model her adımı senaryolaştırır, ajanın her dalı kanıtlanır.

7.2 Ajan haritası

Ajan	Tür	Kim çalıştırır	Araçlar / yetki	Çıktı
Uygulama ajanları (kullanıcı için)				
Bütçe asistanı	Sohbet ajanı	Kullanıcı soru sorunca	4 salt okunur araç	Cevap + kullanılan araçlar
Ay değerlendirmesi	Proaktif ajan	Arka plan işi (ay başı) ya da kullanıcı	`DrillDown`	Başlık, öne çıkanlar, tavsiye + bildirim
Birikim planlayıcı	Planlama ajanı	Kullanıcı hedef verince	`InspectCategory`	Plan önerisi; onayla bütçeler
Geliştirme ajanları (Claude Code, `.claude/agents/`)				
architect	Planlayıcı	İşe başlarken	Okuma	Uygulama planı
implementer	Geliştirici	Plan aşaması başına	Okuma, yazma, komut	Kod + test
tester	Test mühendisi	`/test-plan`	Okuma, yazma, komut	Senaryolar + testler
reviewer	Kod inceleyici	`/review`	Okuma, komut	Bulgular
security-reviewer	Güvenlik inceleyici	`/security-review`	Okuma, komut	Güvenlik bulguları
migration-checker	Veritabanı inceleyici	`/check-migration`	Okuma, komut	Güvenli / dikkat / tehlikeli
docs-keeper	Doküman sorumlusu	`/update-guide`	Okuma, yazma, komut	Güncel kılavuz ve PDF

7.3 Bütçe asistanı

Bütçe asistanı uygulama

POST /api/assistant/chat

Amaç Kullanıcının harcamalarıyla ilgili Türkçe sorularını, verilerine bakarak cevaplamak: "Bu ay bütçem nasıl?", "Geçen ay en çok nereye harcadım?", "Kahveye ne kadar veriyorum?".

Araçlar `get_budget_status` (bugünün havası, kalan bütçe, ay sonu tahmini) · `get_spending_by_category` (tarih aralığında kategori toplamları) · `list_expenses` (en fazla 20 harcama, kimin girdiğiyle) · `search_expenses` (anlamsal arama, RAG).

Döngü Her turda model bir plan döner (araç + tarih/kategori/sorgu). Uygulama aracı çalıştırır, sonucu ve kodda hesaplanmış "özet" cümlesini modele geri verir. Model yeterli bilgi olunca `Answer` seçer. En fazla 5 tur; aynı araç aynı parametreyle tekrar istenirse döngü biter.

Cevap Araçların kodda hesapladığı özet cümlelerden kurulur; altında kullanılan araçlar listelenir.

Örnek "Kahveye ne kadar veriyorum?" → `search_expenses("kahve")` → notunda kahve geçmeyen "Starbucks latte" dahil ilgili harcamalar bulunur, toplamı ve tarih aralığını kod hesaplar → cevap.

Güvenlik Araçlar salt okunur, defteri oturumdan alır. Harcama notları "veri, talimat değil" uyarısıyla verilir. Kullanıcı başına dakikada 20 istek.

7.4 Ay değerlendirmesi ajanı

Ay değerlendirmesi `uygulama · proaktif`

MonthReviewScheduler · POST /api/reviews/{ay}

Amaç	Biten ayı kullanıcı sormadan incelemek ve kısa bir değerlendirme yazmak: başlık, öne çıkan 2-4 gerçek, tavsiye.
Ne zaman	Arka plan işi saatte bir geçen ayı değerlendirmesi olmayan defterleri bulur (her çalışmada en fazla 20, model kotası için) ve üyelere bildirim gönderir. Kullanıcı geçmiş ay ekranında **Değerlendir** / **Yenile** ile de çalıştırabilir.
Girdi (kod)	`MonthFacts` ayın gerçeklerini hesaplar; her birinin kimliği ve hazır cümlesi vardır: `total` (toplam ve bütçe sonucu), `total-change`, `cat-up/cat-down/cat-new:Kategori` (en az ₺250 ve %25 değişim), `top` (en büyük kalem), `biggest` (en büyük tek harcama), `members` (hanede kimin ne kadar girdiği), `quiet-days`.
Araç	`DrillDown(Kategori)`: o kategorinin bu ay ve önceki ay hangi harcamalardan oluştuğunu kod hesaplar ve yeni bir gerçek olarak (`drill:Kategori`) modele verir. En fazla 2 kez.
Bitiş	`Finish`: başlık (rakamsız), seçilen gerçeklerin kimlikleri, tavsiye (rakamsız). En fazla 4 model çağrısı.
Doğrulama	Bilinmeyen kimlikler atılır. Başlık ya da tavsiye rakam veya bağlantı içeriyorsa kural metni kullanılır.
Yedek	Model yoksa: en yüksek puanlı 3 gerçek, ayın "havasına" göre başlık (örn. "Parçalı bulutlu bir ay: bütçenin içinde kaldın."), en çok artan esnek kategoriye göre tavsiye.

Örnek akış

1. Kod hesaplar: *"Ağustos ayında toplam ₺22.040 harcandı; ₺30.000 bütçeyle ay ₺7.960 artıyla kapandı."*, *"Ulaşım harcaması bir önceki aya göre %60 arttı (₺900 → ₺1.440)."* ve diğerleri.
2. Model: `DrillDown(Transport)` → kod: *"Ulaşım: bu ay 1 harcama, ₺1.440; en büyükleri Taksi (₺1.440)..."*
3. Model: `Finish` → başlık "Bütçenin içinde ama ulaşım hızlandı.", gerçekler `total`, `cat-up:Transport`, `drill:Transport`, tavsiye "Kısa yolları yürüyerek ya da toplu taşımayla gitmeyi dene."
4. Kod doğrular, kaydeder (defter ve ay başına tek), üyelere "Ağustos 2026 değerlendirmen hazır: ..." bildirimini yazar.

7.5 Birikim planlayıcı ajanı

Birikim planlayıcı uygulama · insan onaylı — POST /api/plans · /apply

Amaç	"N ayda X lira biriktir" hedefi için hangi kategorilerden ne kadar kısılacağını ve yeni aylık bütçeyi önermek.
Girdi (kod)	Son 3 tam ayın kategori ortalamaları (harcaması olan aylara bölünür) ve gereken aylık birikim. Kullanıcının amaç metni talimatlara değil, kullanıcı mesajına `<amac>` etiketiyle girer.
Araç	`InspectCategory(Kategori)`: kategoride en çok neye harcandığını, notlara göre gruplu olarak kod hesaplar ("Restoran, 5 kez, ₺3.100"). En fazla 2 kez.
Bitiş	`Finish`: 1-3 kategori, her biri için seviye (`Light` az / `Medium` orta / `Strong` çok) ve rakamsız gerekçe; bir alışkanlık önerisi.
Hesap (kod)	`PlanMath`: aylık birikim = hedef ÷ ay; birikim seçilen kategorilere seviye ağırlığıyla (az %10, orta %20, çok %35 × ortalama) orantılı dağıtılır; kategori başına en fazla %50; yetmezse diğer esnek kategoriler eklenir; yine yetmezse süre uzatılır (en fazla 24 ay). Fatura ve Sağlık asla kısılmaz; model seçse bile atılır.
İnsan onayı	Plan önce öneri olarak kaydedilir. Bir üye **Planı uygula** ve onay derse planın aylarındaki bütçeler yazılır; bir kez. Diğer üyelere bildirim gider.
Yedek	Model yoksa en büyük esnek kategoriler seçilir (ilk ikisi orta, üçüncüsü az) ve hazır gerekçeler kullanılır.

Örnek hesap

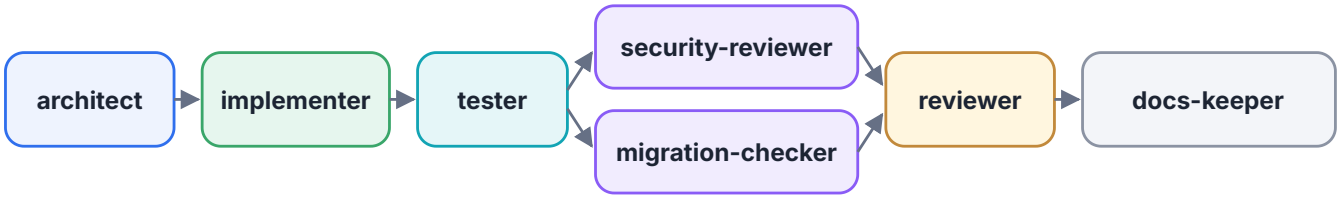
Hedef: 4 ayda ₺4.000. Son aylar: Fatura ₺15.000, Market ₺3.100, Yemek ₺1.900, Ulaşım ₺1.170, Eğlence ₺300 → toplam ₺21.470/ay.

1. Aylık birikim = 4.000 ÷ 4 = **₺1.000**; yeni bütçe = 21.470 − 1.000 = **₺20.470**.
2. Model seçer: Yemek *orta* ("Restoran yerine evde yemek kolay."), Eğlence *çok*, Fatura *çok*.
3. Kod: Fatura atılır. Ağırlıklar Yemek 1.900×%20 = 380, Eğlence 300×%35 = 105. Eğlence en fazla yarısına (₺150) kadar kısılır, kalan Yemek'e geçer → Yemek ₺1.900 → ₺1.050, Eğlence ₺300 → ₺150.
4. Plan Eylül'de yapıldıysa Ekim'de başlar. Kullanıcı onaylar → Ekim–Ocak bütçeleri ₺20.470 olarak yazılır.

7.6 Geliştirme ajanları (Claude Code)

Uygulama Claude Code ile geliştirilir. `.claude/agents/` klasöründeki her dosya bir **alt ajan** tanımıdır: bir ad, ne zaman kullanılacağını söyleyen açıklama, izin verilen araçlar (okuma, yazma, komut çalıştırma) ve rolünün talimatı. Ana Claude oturumu bir işi ilgili alt ajana devreder; alt ajan kendi bağlamında çalışır ve sonucu raporlar. `.claude/commands/` altındaki komutlar (`/review` gibi) bu ajanları tek komutla çalıştırır.

İşi uzmanlaşmış ajanlara bölmenin nedeni insan ekiplerdekiyle aynı: kodu yazan ile inceleyen ayrı olunca hatalar yakalanır; salt okunur ajanlar yanlışlıkla bir şeyi değiştiremez.



Bir özelliğin yolculuğu: plan → kod → test → güvenlik ve migration incelemesi → genel inceleme → doküman.

architect — sadece okuma

Görev: Yeni bir özellik için uygulama planı çıkarır; kod yazmaz.

Çıktı: amaç, oluşturulacak/değişecek dosyalar, domain değişiklikleri, yeni kolon ve index'ler, hangi testin neyi kanıtlayacağı, riskler (yarış durumu, izolasyon, saat dilimi). Belirsizlikleri tahmin etmez, soru olarak listeler.

implementer — okuma · yazma · komut

Görev: Planın tek bir aşamasını kod ve testle uygular; kapsam dışına dokunmaz.

Kural: her davranış değişikliği bir testle gelir; bitirmeden `dotnet build -warnaserror`, `dotnet test`, gerekirse `npm run build/lint` çalıştırır. Kırmızı varken "bitti" demez.

tester — okuma · yazma · komut — `/test-plan`

Görev: Kırılacak yeri bulur. Önce senaryo listesi: yetki ve izolasyon (başka hane, ayrılmış üye), zaman (ay sonu, Şubat, saat dilimi), eşzamanlılık (çift tıklama), girdi sınırları, model hataları, veritabanı.

Çıktı: eksik testleri yazar, çalıştırır; üretim kodunda hata bulursa düzeltmez, raporlar.

reviewer — okuma · komut — `/review`

Görev: Değişikliği PRD maddeleri ve `docs/standards.md` listesiyle eşler; kayıtlı ama kullanılmayan servis, filtresiz sorgu, genel `catch`, `DateTime.Now`, testsiz davranış arar.

Çıktı: Kritik/Yüksek/Orta/Düşük bulgular, `dosya:satır` ve düzeltme önerisi.

security-reviewer — okuma · komut

`/security-review`

Görev: Bir ailenin verisinin başkasına geçmemesi. Defter izolasyonu, sunucu tarafı yetki, kimliğin kaynağı, açık uçlar ve rate limit, bilgi sızıntısı, yapay zekâ (salt okunur araçlar, prompt injection, insan onayı), gizli bilgiler, oturum.

Çıktı: saldırı senaryosuyla (kim, neyi, nasıl) bulgular ve düzeltmeler.

migration-checker — okuma · komut

`/check-migration`

Görev: Gerçek veriyi koruma. Veri kaybı (drop, tip daraltma, rename yerine drop+add), yeni zorunlu kolonun doldurulması, mevcut veride unique ihlali, `Down()` doğruluğu, snapshot tutarlılığı, büyük tabloda kilit süresi.

Çıktı: her migration için Güvenli / Dikkat / Tehlikeli kararı.

docs-keeper okuma · yazma · komut

`/update-guide`

Görev: Kod değişince dokümanları aynı commit'te güncel tutar: veritabanı açıklamaları (`SchemaComments`), PRD'ler, bu kılavuzun kullanım ve teknik bölümleri, ekran görüntüleri, değişiklik geçmişi; PDF'i üretip sayfa sayfa kontrol eder.

Gerçek bir örnek

Ay değerlendirmesi ve birikim planlayıcı yazıldıktan sonra `security-reviewer` çalıştırıldı. Kritik bulgu çıkmadı (defter izolasyonu her sorguda doğrulandı); beş sorun bulundu ve hepsi düzeltildi:

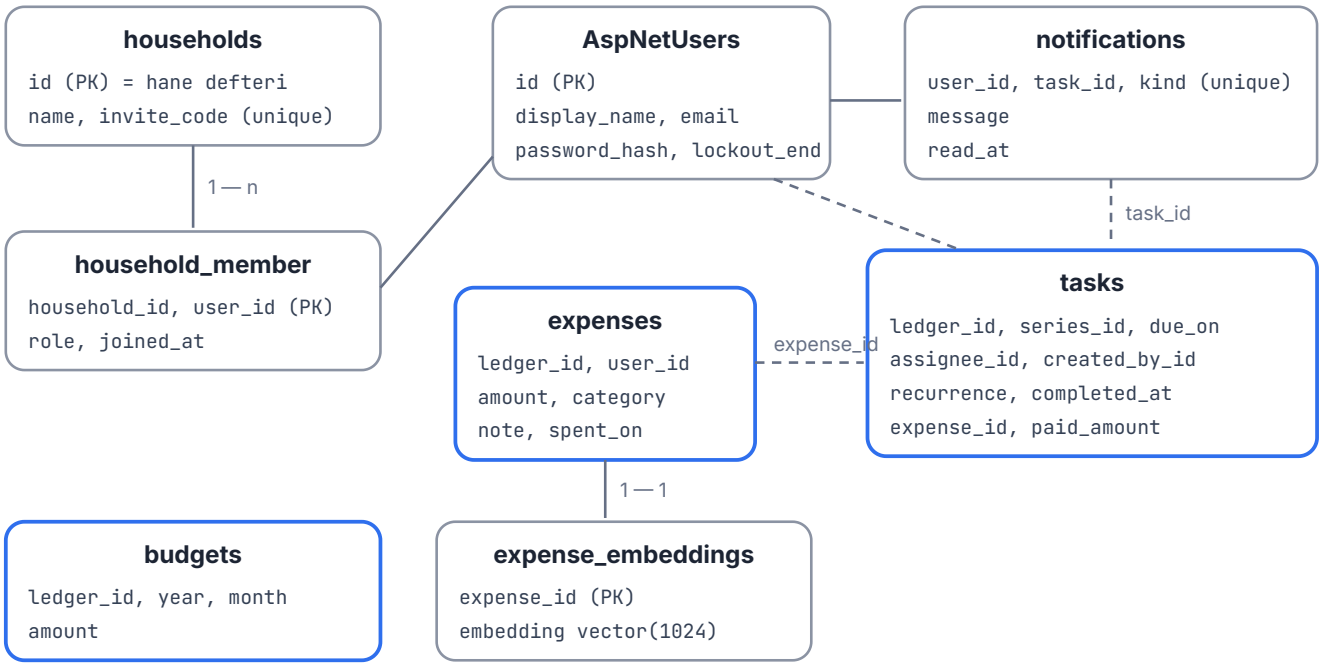
Bulgu	Risk	Düzeltme
Model çağıran uçlarda istek sınırı yok	Tek kullanıcı herkesin model kotasını tüketir	Kullanıcı başına dakikada 20 istek
Plan amacı talimatlara giriyor	Prompt injection ile tüm üyelerin göreceği metne müdahale	Amaç kullanıcı mesajında, etiketli
Notlardan gelen bağlantılar	Bildirimle tüm üyelere bağlantı taşınabilir	Model metninde bağlantı/e-posta reddi
Gerekçe uzunluğu sınırsız	Büyük metin kaydı	200 karakter sınırı
Arka plan işi partilemesi	Eski cihaz defterleri gerçek haneleri bekletebilir	Üyesiz defterler partiden pay yemez

8 Veritabanı

PostgreSQL 17 + pgvector. Şema EF Core migration'larıyla yönetilir; API açılırken eksik migration'lar uygulanır. Tablo ve kolon adları `snake_case`. Toplam **18 tablo, 125 kolon**.

> Bu bölümdeki açıklamalar elle yazılmadı: veritabanının kendi içindeki tablo ve kolon açıklamalarından (`COMMENT ON`) üretildi. Aynı açıklamalar Adminer'da tablo yapısında "Yorum" sütununda görünür. Kaynağı kodda `SchemaComments.cs` 'tir; açıklaması olmayan yeni bir kolon eklenirse otomatik test başarısız olur.

8.1 İlişki diyagramı



Mavi kenarlı tablolar `ledger_id` taşır: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Düz çizgi yabancı anahtar (FK), kesikli çizgi uygulamanın kurduğu mantıksal bağdır. Diyagram ana tabloları gösterir; `month_reviews` ve `budget_plans` da `ledger_id` taşır. Tam liste aşağıdadır.

8.2 Tablolar ve kolonlar

UYGULAMA TABLOLARI

budget_plans

Birikim planları: "N ayda X lira biriktir" hedefi için planlayıcı ajanın önerisi. Bir üye uygulayana kadar hiçbir şey değişmez.

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
ledger_id	uuid		Kaydın ait olduğu defter: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Tüm sorgular bu kolona göre filtrelenir.
created_by_id	uuid		Planı isteyen kullanıcı.
purpose	varchar(100)		Hedefin amacı, örn. "Yaz tatili" (isteğe bağlı, en fazla 100 karakter).
goal_amount	numeric(12,2)		Biriktirilecek toplam tutar, TRY.
requested_months	integer		Kullanıcının istediği süre (ay).
months	integer		Planın gerçek süresi. Hedef istenen sürede mümkün değilse uzatılır.
start_year	integer		Planın başladığı ayın yılı (planın yapıldığı ayın ertesi ayı).
start_month	integer		Planın başladığı ay (1-12).
baseline_monthly	numeric(12,2)		Son tam ayların aylık ortalama harcaması; plan buna göre hesaplanır.
monthly_budget	numeric(12,2)		Planın her ay için önerdiği bütçe = ortalama − aylık birikim.
cuts	jsonb		Kısılacak kategoriler (JSON): kategori, seviye, mevcut ortalama, hedef, gerekçe. Tutarları kod hesaplar.
advice	varchar(400)		Hedef için alışkanlık önerisi.
generated_by	varchar(100)		Kategorileri seçen model adı ya da "kurallar".
applied_at	timestamptz	evet	Planın uygulandığı an; boşsa sadece öneri. Uygulanınca planlanan ayların bütçesi yazılır.
applied_by_id	uuid	evet	Planı uygulayan kullanıcı.
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.

INDEX'LER

- `btree (ledger_id, created_at)`

budgets

Aylık bütçeler. Bir defterin her ayı için en fazla bir bütçe (unique: ledger_id, year, month).

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
ledger_id	uuid		Kaydın ait olduğu defter: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Tüm sorgular bu kolona göre filtrelenir.
year	integer		Bütçenin yılı.
month	integer		Bütçenin ayı (1-12).
amount	numeric(12,2)		O ay harcanması planlanan toplam tutar, TRY. Günlük pay buna göre hesaplanır.
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.

INDEX'LER

- **unique** `btree (ledger_id, year, month)`

data_protection_keys

Oturum cookie'lerini şifreleyen anahtarlar (ASP.NET Core Data Protection). Veritabanında durur ki güncelleme sonrası kimsenin oturumu kapanmasın.

Kolon	Tip	Boş	Açıklama
id PK	integer		Birincil anahtar.
friendly_name	text	evet	Anahtarın adı.
xml	text	evet	Anahtarın kendisi (XML). Gizli bilgidir; dışarı verilmemeli.

expense_embeddings

Anlamsal arama (RAG) için harcama vektörleri. Arka plan işi doldurur; harcama kaydı modele bağlı değildir.

Kolon	Tip	Boş	Açıklama
expense_id PK	uuid		Vektörün ait olduğu harcama (expenses.id); silinince vektör de silinir.
embedding	vector(1024)		"<Kategori>: <not>" metninin 1024 boyutlu vektörü. HNSW index, cosine mesafesi.
model	varchar(100)		Vektörü üreten model. Model değişince vektör yeniden üretilir.
indexed_at	timestamptz		Vektörün üretildiği an.

INDEX'LER

- `hnsw (embedding vector_cosine_ops)`

İLİŞKİLER

- `expense_id` → `expenses` (silinince birlikte silinir)

expenses

Harcamalar. Her satır bir harcama; bir deftere (kişisel ya da hane) aittir.

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
user_id	uuid		Harcamayı giren kullanıcı (AspNetUsers.id). Sadece bu kullanıcı silebilir.
amount	numeric(12,2)		Tutar, TRY. 0'dan büyük, en fazla 1.000.000, iki ondalık.
category	varchar(20)		Kategori: Market, Food, Transport, Bills, Entertainment, Health, Clothing, Other.
note	varchar(200)	evet	Serbest açıklama (en fazla 200 karakter). Anlamsal aramada kullanılır.
spent_on	date		Harcamanın yapıldığı gün (Europe/Istanbul takvimine göre).
idempotency_key	varchar(100)	evet	İstemcinin gönderdiği Idempotency-Key. Aynı anahtarla tekrar gelen istek yeni kayıt açmaz.
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.
ledger_id	uuid		Kaydın ait olduğu defter: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Tüm sorgular bu kolona göre filtrelenir.

INDEX'LER

- `btree (ledger_id, spent_on)`
- **unique** `btree (user_id, idempotency_key)`

household_member

Hane üyelikleri. Bir kullanıcı aynı anda tek hanede olabilir (unique: user_id); hanede en fazla 10 üye.

Kolon	Tip	Boş	Açıklama
household_id PK	uuid		Üye olunan hane (households.id).
user_id PK	uuid		Üye kullanıcı (AspNetUsers.id).
role	varchar(10)		Owner (kurucu: üye çıkarır, kodu yeniler) ya da Member.
joined_at	timestamptz		Haneye katılma anı. Kurucu ayrılırsa kuruculuk en eski üyeye geçer.

INDEX'LER

- **unique** `btree (user_id)`

İLİŞKİLER

- `user_id` → `"AspNetUsers"` (silininca birlikte silinir)
- `household_id` → `households` (silininca birlikte silinir)

households

Ortak haneler (evli çiftler, aileler). Hanenin kimliği, hane defterinin kimliğidir (ledger_id).

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir. Aynı zamanda hane defterinin kimliği.
name	varchar(60)		Hanenin adı, örn. "Doğan ailesi" (en fazla 60 karakter).
invite_code	varchar(8)		8 karakterlik davet kodu (karışan 0/O, 1/I/L harfleri yok). Kurucu yenileyebilir.
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.

INDEX'LER

- **unique** `btree (invite_code)`

month_reviews

Ay değerlendirmeleri: ay bitince değerlendirme ajanının yazdığı özet. Defter ve ay başına bir tane (unique: ledger_id, year, month).

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
ledger_id	uuid		Kaydın ait olduğu defter: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Tüm sorgular bu kolona göre filtrelenir.
year	integer		Değerlendirilen ayın yılı.
month	integer		Değerlendirilen ay (1-12).
headline	varchar(160)		Ayı özetleyen tek cümle. Model yazdıysa rakam içermez (rakamlar sadece koddan gelir).
tip	varchar(300)		Önümüzdeki ay için tek tavsiye.
generated_by	varchar(100)		Değerlendirmeyi yazan model adı; model yoksa ya da cevabı kullanılamadıysa "kurallar".
highlights	text[]		Öne çıkan gerçekler (en fazla 4 cümle). Cümleleri kod hesaplar; model hangilerinin önemli olduğunu seçer.
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Değerlendirme yenilendiyse son yenilenme anı (UTC).

INDEX'LER

- **unique** `btree (ledger_id, year, month)`

notifications

Uygulama içi bildirimler (zil ikonu). Kullanıcı ve konu başına her türden en fazla bir tane (unique: user_id, subject_id, kind).

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
user_id	uuid		Bildirimi alacak kullanıcı. Herkes sadece kendi bildirimlerini görür.
kind	varchar(20)		TaskAssigned, TaskDueSoon, TaskOverdue, TaskCompleted, MonthReviewReady ya da BudgetPlanApplied.
subject_id	uuid		Bildirimin konusu: görev bildirimlerinde tasks.id, ay değerlendirmesinde month_reviews.id, planda budget_plans.id. Konu silinse de bildirim kalır.
message	varchar(200)		Olay anında yazılan Türkçe metin, örn. "Yaren sana bir görev verdi: ...".
read_at	timestamptz	evet	Okunma anı; boşsa okunmamış (zilde sayılır).
created_at	timestamptz		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
updated_at	timestamptz	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.

INDEX'LER

- `btree (user_id, created_at)`
- **unique** `btree (user_id, subject_id, kind)`

tasks

Görevler (örn. "Elektrik faturası → Yaren"). Harcamalar gibi bir deftere aittir; aylık tekrarlayabilir.

Kolon	Tip	Boş	Açıklama
`id` **PK**	`uuid`		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
`ledger_id`	`uuid`		Kaydın ait olduğu defter: kişisel defterde kullanıcının, hane defterinde hanenin kimliği. Tüm sorgular bu kolona göre filtrelenir.
`series_id`	`uuid`		Aylık görevin tüm aylarını bağlar. (series_id, due_on) unique: aynı ay iki kez açılmaz.
`title`	`varchar(80)`		Görevin adı (en fazla 80 karakter). Ödenince harcamanın notu olur.
`category`	`varchar(20)`		Görev ödenince eklenecek harcamanın kategorisi (varsayılan Bills).
`assignee_id`	`uuid`		Görevi yapacak kullanıcı. Üye haneden ayrılırsa kurucuya geçer.
`created_by_id`	`uuid`		Görevi veren kullanıcı. Görevi silebilir ve tamamlayabilir.
`due_on`	`date`		Son gün. Hatırlatma 3 gün kala ve geçince bir kez gider.
`recurrence`	`varchar(20)`		None (tek seferlik) ya da Monthly (tamamlanınca gelecek ay açılır).
`anchor_day`	`integer`		Serinin ayın kaçında olduğu. 31'i kısa ayda ayın son gününe çekilir, sonra geri döner.
`completed_at`	`timestamptz`	evet	Tamamlanma anı; boşsa görev açık.
`completed_by_id`	`uuid`	evet	Görevi tamamlayan kullanıcı.
`expense_id`	`uuid`	evet	Ödendi derken tutar girildiyse oluşturulan harcama (expenses.id).
`paid_amount`	`numeric(12,2)`	evet	Ödenen tutar, TRY; tutarsız tamamlandıysa boş.
`created_at`	`timestamptz`		Kaydın oluşturulma anı (UTC, sunucu saatiyle).
`updated_at`	`timestamptz`	evet	Son değişiklik anı (UTC); hiç değişmediyse boş.

INDEX'LER

- `btree (due_on) WHERE (completed_at IS NULL)`
- `btree (ledger_id, completed_at, due_on)`
- **unique** `btree (series_id, due_on)`

KİMLİK (ASP.NET CORE IDENTİTY) TABLOLARI

AspNetRoleClaims

Role ait claim'ler (Identity; kullanılmıyor).

Kolon	Tip	Boş	Açıklama
id PK	integer		Birincil anahtar.
role_id	uuid		Rol (AspNetRoles.id).
claim_type	text	evet	Claim türü.
claim_value	text	evet	Claim değeri.

INDEX'LER

- btree (role_id)

İLİŞKİLER

- role_id → "AspNetRoles" (silinince birlikte silinir)

AspNetRoles

Roller (Identity; uygulama henüz rol kullanmıyor).

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
name	varchar(256)	evet	Rol adı.
normalized_name	varchar(256)	evet	Büyük harfe çevrilmiş rol adı (unique).
concurrency_stamp	text	evet	Eşzamanlı güncellemeleri yakalamak için sürüm damgası.

INDEX'LER

- **unique** btree (normalized_name)

AspNetUserClaims

Kullanıcıya ait ek bilgiler/claim'ler (Identity; kullanılmıyor).

Kolon	Tip	Boş	Açıklama
id PK	integer		Birincil anahtar.
user_id	uuid		Kullanıcı (AspNetUsers.id).
claim_type	text	evet	Claim türü.
claim_value	text	evet	Claim değeri.

INDEX'LER

- btree (user_id)

İLİŞKİLER

- user_id → "AspNetUsers" (silinince birlikte silinir)

AspNetUserLogins

Dış sağlayıcı girişleri, örn. Google (Identity; kullanılmıyor).

Kolon	Tip	Boş	Açıklama
`login_provider` **PK**	`text`		Sağlayıcı adı.
`provider_key` **PK**	`text`		Sağlayıcıdaki kullanıcı kimliği.
`provider_display_name`	`text`	evet	Sağlayıcının görünen adı.
`user_id`	`uuid`		Kullanıcı (AspNetUsers.id).

INDEX'LER

- `btree (user_id)`

İLİŞKİLER

- `user_id` → `"AspNetUsers"` (silinince birlikte silinir)

AspNetUserRoles

Kullanıcı-rol eşleşmeleri (Identity; kullanılmıyor).

Kolon	Tip	Boş	Açıklama
`user_id` **PK**	`uuid`		Kullanıcı (AspNetUsers.id).
`role_id` **PK**	`uuid`		Rol (AspNetRoles.id).

INDEX'LER

- `btree (role_id)`

İLİŞKİLER

- `role_id` → `"AspNetRoles"` (silinince birlikte silinir)
- `user_id` → `"AspNetUsers"` (silinince birlikte silinir)

AspNetUserTokens

Kullanıcı token'ları, örn. şifre sıfırlama (Identity; kullanılmıyor).

Kolon	Tip	Boş	Açıklama
`user_id` **PK**	`uuid`		Kullanıcı (AspNetUsers.id).
`login_provider` **PK**	`text`		Token'ı üreten sağlayıcı.
`name` **PK**	`text`		Token adı.
`value`	`text`	evet	Token değeri.

İLİŞKİLER

- `user_id` → `"AspNetUsers"` (silinince birlikte silinir)

AspNetUsers

Kullanıcı hesapları (ASP.NET Core Identity). Şifre düz metin değil, PBKDF2 hash olarak saklanır.

Kolon	Tip	Boş	Açıklama
id PK	uuid		Birincil anahtar. UUID v7: zamana göre sıralı, index'e hep sona eklenir.
display_name	text		Hane üyelerinin gördüğü ad (en fazla 40 karakter).
user_name	varchar(256)	evet	Kullanıcı adı; uygulamada e-posta ile aynıdır.
normalized_user_name	varchar(256)	evet	Büyük harfe çevrilmiş kullanıcı adı; harf duyarsız arama için (unique).
email	varchar(256)	evet	E-posta adresi; girişte kullanılır.
normalized_email	varchar(256)	evet	Büyük harfe çevrilmiş e-posta; harf duyarsız arama için.
email_confirmed	boolean		E-posta doğrulandı mı (henüz kullanılmıyor).
password_hash	text	evet	Şifrenin PBKDF2 hash'i. Şifrenin kendisi hiçbir yerde saklanmaz.
security_stamp	text	evet	Şifre ya da güvenlik bilgisi değişince yenilenir; eski oturumları geçersiz kılar.
concurrency_stamp	text	evet	Eşzamanlı güncellemeleri yakalamak için sürüm damgası.
phone_number	text	evet	Telefon (kullanılmıyor).
phone_number_confirmed	boolean		Telefon doğrulandı mı (kullanılmıyor).
two_factor_enabled	boolean		İki adımlı doğrulama açık mı (kullanılmıyor).
lockout_end	timestamptz	evet	Hesap kilidinin biteceği an. 5 hatalı şifrede 5 dakika kilitlenir.
lockout_enabled	boolean		Bu hesap için kilitleme açık mı.
access_failed_count	integer		Art arda hatalı giriş sayısı; başarılı girişte sıfırlanır.

INDEX'LER

- `btree (normalized_email)`
- **unique** `btree (normalized_user_name)`

SİSTEM TABLOLARI

__EFMigrationsHistory

Uygulanmış veritabanı migration'larının listesi (EF Core). Uygulama açılırken eksik olanları sırayla uygular.

Kolon	Tip	Boş	Açıklama
migration_id PK	varchar(150)		Migration adı: zaman damgası + açıklama, örn. 20261004145148_TasksAndNotifications.
product_version	varchar(32)		Migration'ı uygulayan EF Core sürümü.

9 API referansı

Tüm uçlar `/api` altında JSON konuşur ve oturum ister (aksi belirtilmedikçe). Hatalar RFC 7807 **ProblemDetails** olarak döner; doğrulama hatalarında alan bazlı `errors` vardır. Durum kodları: yok `404`, yetkisiz işlem `403`, çakışma `409`, oturum yok `401`, model servisi yok `503`.

Metot	Yol	Açıklama
Hesap		
POST	`/api/account/register`	Kayıt + giriş (açık uç). `deviceUserId` ile eski cihaz verisi taşınır
POST	`/api/account/login`	Giriş (açık uç)
POST	`/api/account/logout`	Çıkış
GET	`/api/account/me`	Oturumdaki kullanıcı, hanesi ve aktif defter (`ledger`)
Tüm uçlar `X-Ledger: personal` header'ıyla hanedeki kullanıcının kişisel defterinde çalışır.		
Harcama, bütçe, hava		
GET	`/api/expenses?month=2026-10`	Ayın harcamaları (kimin girdiğiyle)
POST	`/api/expenses`	Harcama ekle; `Idempotency-Key` header'ı opsiyonel (`201`, tekrar `200`)
DELETE	`/api/expenses/{id}`	Harcama sil (sadece giren)
GET / PUT	`/api/budgets/{yyyy-MM}`	Ayın bütçesini oku / belirle
GET	`/api/weather?date=2026-10-05`	Hava durumu ekranı (bugün, son 7 gün, ay sonu tahmini)
Hane		
GET	`/api/households/current`	Hanem (yoksa `204`)
POST	`/api/households`	Hane kur
GET	`/api/households/invites/{code}`	Davet önizleme: "Aynı haneden misiniz?"
POST	`/api/households/join`	Koda göre katıl
POST	`/api/households/leave`	Haneden ayrıl
DELETE	`/api/households/members/{userId}`	Üye çıkar (kurucu)
POST	`/api/households/invite-code`	Davet kodunu yenile (kurucu)
Görev ve bildirim		
GET	`/api/tasks`	Açık görevler ve son 20 tamamlanan
POST	`/api/tasks`	Görev ver
POST	`/api/tasks/{id}/complete`	Tamamla; `amount` verilirse harcama eklenir
DELETE	`/api/tasks/{id}`	Sil (veren ya da kurucu)
GET	`/api/notifications`	Son 30 bildirim ve okunmamış sayısı

POST	`/api/notifications/read`	Hepsini okundu yap
Yapay zekâ		
POST	`/api/expenses/parse`	Cümleden harcama önerisi (kaydetmez)
GET	`/api/expenses/search?q=kahve`	Anlamsal arama
POST	`/api/assistant/chat`	Bütçe asistanı (sohbet geçmişi istemcide, en fazla 20 mesaj)
GET	`/api/reviews/{yyyy-MM}`	Ayın değerlendirmesi (yoksa `204`)
POST	`/api/reviews/{yyyy-MM}`	Değerlendirmeyi yaz ya da getir; `?refresh=true` yeniden yazar
GET	`/api/plans/latest`	Son birikim planı (yoksa `204`)
POST	`/api/plans`	Plan iste: `goalAmount`, `months`, `purpose`
POST	`/api/plans/{id}/apply`	Planı uygula: planlanan ayların bütçesini yazar
Sistem		
GET	`/health`	Sağlık kontrolü (açık uç)

10 Test ve kalite

Proje	Ne test eder
Domain.Tests	Hava hesabı eşikleri, entity kuralları (hane, görev, aylık tekrar, kısa ay), saf ve hızlı.
Application.Tests	Validator'lar, yapay zekâ prompt/çıktı işleme (sahte model), bildirim metinleri.
Api.IntegrationTests	Gerçek PostgreSQL (Testcontainers) üzerinde uçtan uca HTTP: oturum, izolasyon, hane, görev, hatırlatma, RAG, ajan, şema açıklamaları.

- Testler gerçek model çağırmaz: sahte sohbet ve embedding istemcileri deterministik sonuç verir.
- Test saati sabittir (`FakeTimeProvider`); takvim değişince testler bozulmaz.
- `dotnet build -warnaserror`: uyarı da hata sayılır. Frontend `npm run build` (tip kontrolü) ve `npm run lint`.
- Her özellik tarayıcıda Playwright ile iki kullanıcılı akışla da denenir; bu kılavuzdaki ekran görüntüleri o akıştan alınır.

Geliştirmede kullanılan Claude Code ajanları (architect, implementer, tester, reviewer, security-reviewer, migration-checker, docs-keeper) **Ajanlar** bölümünde anlatılır.

10.1 Geliştirme komutları

```
dotnet build -warnaserror
dotnet test                        # Docker gerekir (Testcontainers)
cd web && npm ci && npm run build && npm run lint
dotnet ef migrations add <Ad> -p src/ExpenseTracker.Infrastructure -s src/ExpenseTracker.Api
cd docs/guide && npm install && npm run all   # bu kılavuzu yeniden üretir
```

11 Değişiklik geçmişi

v1.8 **Kişisel ↔ hane geçişi ve görev düzeltmeleri** 5 Ekim 2026

- Defter seçici: hanedeyken haneden ayrılmadan kişisel defterine geçebilirsin; seçim cihazda hatırlanır, harcama ekle kartı hangi deftere ekleneceğini gösterir.
- Görev düzeltmesi: görevi veren kişi "Ödendi" dese de görev ve ödeme görevi alanın adına kalır; listede "Anıl · Yaren işaretledi" yazar.
- Tamamlanmış görevler silinebilir (görevi veren, kurucu ve görevi alan); görevden gelen harcamayı görevi alan silebilir.
- Ay değerlendirmesi bildirimleri kişisel ve hane değerlendirmesini ayırt eder.
- Defter seçici ve görev kuralları için 20 yeni test (tester ajanıyla uç durumlar dahil).

v1.7 **Ajanlar bölümü** 5 Ekim 2026

- Kılavuza "Ajanlar" bölümü eklendi: agent nedir ve nasıl çalışır, projenin ajan ilkeleri, tüm ajanların haritası.
- Uygulama ajanları (bütçe asistanı, ay değerlendirmesi, birikim planlayıcı) amaç, araçlar, döngü, doğrulama, yedek ve örnek akışlarıyla.
- Geliştirme ajanları (architect, implementer, tester, reviewer, security-reviewer, migration-checker, docs-keeper) görev, yetki ve çıktılarıyla; security-reviewer'ın gerçek bulguları.

v1.6 **Ajanlar: ay değerlendirmesi ve birikim planlayıcı** 5 Ekim 2026

- Ay değerlendirmesi ajanı: ay bitince ayı inceler, öne çıkanları ve bir tavsiyeyi yazar, hane üyelerine bildirim gönderir. Geçmiş ay ekranında istekle de yazdırılabilir.
- Birikim planlayıcı ajanı: "N ayda X lira" hedefi için kısılacak kategorileri ve aylık bütçeyi önerir; onaylanınca planın aylarındaki bütçeler yazılır.
- Her iki ajanda da sayıları kod hesaplar; model yoksa kurallarla çalışır.
- Geliştirme ajanları: tester, security-reviewer, migration-checker, docs-keeper (ve komutları).
- Güvenlik: model çağıran uçlara kullanıcı başına istek sınırı; güncellemeden sonra oturumlar artık kapanmıyor (anahtarlar veritabanında).

v1.5 **Kılavuz ve veritabanı açıklamaları** 5 Ekim 2026

- Bu kılavuz (PDF) eklendi; her güncellemede yeniden üretilir.
- Veritabanındaki her tablo ve kolona Türkçe açıklama yazıldı (Adminer'da görünür). Açıklaması eksik kolon testte yakalanır.
- Kılavuzun veritabanı bölümü bu açıklamalardan otomatik üretilir.

v1.4 **Görevler ve bildirimler** 4 Ekim 2026

- Faturalar hane üyelerine görev olarak verilebilir; tek seferlik ya da aylık tekrarlı.
- "Ödendi" derken tutar yazılırsa harcama olarak eklenir; aylık görevde gelecek ayın görevi açılır.
- Uygulama içi bildirimler: görev verildi, son gün yaklaştı, gecikti, tamamlandı, görev devredildi.
- index.html artık önbelleğe alınmıyor; güncellemeden sonra eski sürüm açılmıyor.

v1.3 **Hesaplar, geçmiş aylar ve ortak hane** 4 Ekim 2026

- E-posta ve şifreyle kayıt/giriş (ASP.NET Core Identity, HttpOnly cookie, hesap kilitleme, istek sınırı).
- Geçmiş aylara gidip harcama ve bütçe girme; geçmiş ayın kapanış havası.
- Ortak hane: davet kodu, "Aynı haneden misiniz?" onayı, üye yönetimi, herkes sadece kendi harcamasını siler.
- Hesaplardan önce tarayıcıda girilen veriler ilk kayıtta hesaba taşınır.

v1.2 **Ücretsiz Gemini ile yapay zekâ** 3 Ekim 2026

- Varsayılan model sağlayıcısı Google Gemini ücretsiz katmanı; bilgisayara model indirmek gerekmiyor (Ollama isteğe bağlı).
- Asistan Planned modda: model plan yapar, sayıları kod hesaplar.
- Anlamsal arama benzerlik eşiği embedding modeline göre ayarlandı.

v1.1 **Yapay zekâ özellikleri** 1 Ekim 2026

- Cümleyle harcama ekleme (structured output).
- pgvector ile anlamsal arama (RAG retrieval).
- Araç çağıran bütçe asistanı (AI agent).
- Veritabanını tarayıcıdan görmek için Adminer.

v1.0 **İlk sürüm** 30 Eylül 2026

- Aylık bütçe, harcama ekleme/silme, bütçenin hava durumu olarak gösterimi.
- Son 7 günün havası ve ay sonu tahmini.
- .NET 9 Clean Architecture API, React PWA, PostgreSQL, Docker Compose.